

Lab 9

Stephanie Salgado

Step 1: Installing, configuring and running Snort

```
stephanie@ubuntu:~$ sudo apt-get install snort
[sudo] password for stephanie:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

To begin, I used “sudo apt-get install snort” to install it.

```
# Step #1: Set the network variables. For more information, see README.variables
#####

# Setup the network addresses you are protecting
#
# Note to Debian users: this value is overridden when starting
# up the Snort daemon through the init.d script by the
# value of DEBIAN_SNORT_HOME_NET s defined in the
# /etc/snort/snort.debian.conf configuration file
#
ipvar HOME_NET any

# Set up the external network addresses. Leave as "any" in most situations
ipvar EXTERNAL_NET any
# If HOME_NET is defined as something other than "any", alternative, you can
# use this definition if you do not want to detect attacks from your internal
# IP addresses:
#ipvar EXTERNAL_NET !$HOME_NET

# List of DNS servers on your network
```

Next, I used “sudo nano /etc/snort/snort.conf” to read the comments and find the definitions for variables “HOME_NET” and “EXTERNAL_NET”.

```
stephanie@ubuntu:~$ sudo snort -T -c /etc/snort/snort.conf
[sudo] password for stephanie:
Running in Test mode

--== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "/etc/snort/snort.conf"
```

```
Total snort Fixed Memory Cost - MaxRss:106532
Snort successfully validated the configuration!
Snort exiting
```

I used “sudo snort -T -c /etc/snort/snort.conf” to test the configuration.

```
stephanie@ubuntu:~$ sudo snort
Running in packet dump mode

--== Initializing Snort ==--
```

```
stephanie@ubuntu:~$ ping www.google.com
PING www.google.com (142.251.116.104) 56(84) bytes of data.
64 bytes from rt-in-f104.1e100.net (142.251.116.104): icmp_seq=1
.9 ms
64 bytes from rt-in-f104.1e100.net (142.251.116.104): icmp_seq=2
.9 ms
64 bytes from rt-in-f104.1e100.net (142.251.116.104): icmp_seq=3
.7 ms
64 bytes from rt-in-f104.1e100.net (142.251.116.104): icmp_seq=4
.6 ms
64 bytes from rt-in-f104.1e100.net (142.251.116.104): icmp_seq=5
.2 ms
^C
--- www.google.com ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4006ms
rtt min/avg/max/mdev = 21.938/27.068/34.930/4.745 ms
stephanie@ubuntu:~$
```

```
WARNING: No preprocessors configured for policy 0.
11/14-12:39:42.315799 192.168.200.2:53 -> 192.168.200.128:39698
UDP TTL:128 TOS:0x0 ID:27306 IpLen:20 DgmLen:183
Len: 155
==+=====+

WARNING: No preprocessors configured for policy 0.
11/14-12:39:42.316252 192.168.200.128 -> 142.251.116.104
ICMP TTL:64 TOS:0x0 ID:51924 IpLen:20 DgmLen:84 DF
Type:8 Code:0 ID:9319 Seq:1 ECHO
==+=====+

WARNING: No preprocessors configured for policy 0.
11/14-12:39:42.338185 142.251.116.104 -> 192.168.200.128
ICMP TTL:128 TOS:0x0 ID:27307 IpLen:20 DgmLen:84
Type:0 Code:0 ID:9319 Seq:1 ECHO REPLY
==+=====+

11/14-12:39:42.338521 192.168.200.128:35173 -> 192.168.200.2:53
UDP TTL:64 TOS:0x0 ID:61661 IpLen:20 DgmLen:85
Len: 57
==+=====+
```

Then, I use “sudo snort” and in another terminal I use “ping www.google.com”. Notice Snort is able to capture these packets. This is its “sniffing” mode.

```

stephanie@ubuntu:~$ sudo snort -q -A console -c /etc/snort/snort.conf
11/14-13:42:33.468331  [**] [1:1000001:0] Stephanie Salgado... IP packet is detected. [**] [Priority: 0] {UDP} 192.168.200.1:57621 -> 192.168.200.255:57621
11/14-13:42:37.026006  [**] [1:1000001:0] Stephanie Salgado... IP packet is detected. [**] [Priority: 0] {UDP} 192.168.200.2:53 -> 192.168.200.128:42545
11/14-13:42:37.026011  [**] [1:1000001:0] Stephanie Salgado... IP packet is detected. [**] [Priority: 0] {UDP} 192.168.200.2:53 -> 192.168.200.128:56171
11/14-13:42:37.026534  [**] [1:1000001:0] Stephanie Salgado... IP packet is detected. [**] [Priority: 0] {ICMP} 192.168.200.128 -> 142.250.115.103
11/14-13:42:37.065909  [**] [1:1000001:0] Stephanie Salgado... IP packet is detected. [**] [Priority: 0] {ICMP} 142.250.115.103 -> 192.168.200.128

```

```

stephanie@ubuntu:~$ ping www.google.com
PING www.google.com (142.250.115.103) 56(84) bytes of data.
64 bytes from rq-in-f103.1e100.net (142.250.115.103): icmp_seq=1 ttl=128 time=39.4 ms
64 bytes from rq-in-f103.1e100.net (142.250.115.103): icmp_seq=2 ttl=128 time=36.5 ms

```

This time I use “sudo snort -c /etc/snort/snort.conf” to run Snort in IDS mode, while still using “ping www.google.com”. Notice this time the output looks different. It now shows the alert I added in my own “local.rules”. The process of adding rules and editing the “local.rules” file is shown below in step 2.

Step 2: Adding rules

```

stephanie@ubuntu:~$ sudo nano /etc/snort/rules/local.rules
stephanie@ubuntu:~$ sudo cat /etc/snort/rules/local.rules
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
# -----
# LOCAL RULES
# -----
# This file intentionally does not come with signatures.  Put your local
# additions here.
alert ip any any -> any any (msg: "Stephanie Salgado... IP packet is detected.";
sid:1000001;)

```

I use “sudo nano /etc/snort/rules/local.rules” and add the following rule: alert ip any any -> any any (msg: “Stephanie Salgado... IP packet is detected.”;sid:1000001;). It’s important to mention, I did this while I was completing step 1, but I included it here since it’s more relevant to this section.


```

stephanie@ubuntu:~$ sudo cat /etc/snort/rules/test.rules
#This was modified by Stephanie Salgado.

alert icmp $EXTERNAL_NET any -> $HOME_NET any (sid:385;rev:3;msg:"Stephanie Salgado... ICMP traceroute.")

alert icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"Stephanie Salgado... ICMP traffic inbound.";sid:1; rev:1;)

alert icmp $EXTERNAL_NET any -> $HOME_NET any (sid:385;rev:3;msg:"Stephanie Salgado... ICMP traceroute.";ttl:1;itype:8;reference:arachnids,118;classtype:attempted-recon;)

```

```

GNU nano 7.2 /etc/snort/snort.conf *
# Note: These rules are disabled by default as they are
# too coarse grained. Enabling them causes a large
# performance impact
#include $RULE_PATH/shellcode.rules
include $RULE_PATH/smtp.rules
include $RULE_PATH/snmp.rules
#include $RULE_PATH/specific-threats.rules
#include $RULE_PATH/spyware-put.rules
include $RULE_PATH/sql.rules
include $RULE_PATH/telnet.rules
include $RULE_PATH/test.rules
include $RULE_PATH/tftp.rules
include $RULE_PATH/virus.rules

```

I put the provided “test.rules” file in the “rules” directory. Then, I edited the “snort.conf” file to add the rules to my Snort configuration.

```

stephanie@ubuntu:~$ sudo snort -T -c /etc/snort/snort.conf
Running in Test mode

```

```

Total snort Fixed Memory Cost - MaxRss:104156
Snort successfully validated the configuration!
Snort exiting

```

```

stephanie@ubuntu:~$ sudo /etc/init.d/snort restart
* Stopping Network Intrusion Detection System snort [ OK ]
* Starting Network Intrusion Detection System snort [ OK ]
stephanie@ubuntu:~$ sudo /etc/init.d/snort status
* Status of snort daemon(s) [ OK ]

```

After all this, I tested the configuration, once it was validated, I used “sudo /etc/init.d/snort restart” to make the changes take effect. Then I used “sudo /etc/init.d/snort status” to view the status of Snort. Notice it’s “OK”.

```

stephanie@ubuntu:~$ sudo nano /etc/snort/rules/test.rules
stephanie@ubuntu:~$ sudo cat /etc/snort/rules/test.rules
#This was modified by Stephanie Salgado.

alert icmp $EXTERNAL_NET any -> $HOME_NET any (sid:385;rev:3;msg:"Stephanie Salgado... ICMP traceroute.")

alert icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"Stephanie Salgado... ICMP traffic inbound.";sid:1; rev:1;)

alert icmp $EXTERNAL_NET any -> $HOME_NET any (sid:385;rev:3;msg:"Stephanie Salgado... ICMP traceroute.";ttl:1;itype:8;reference:arachnids,118;classtype:attempted-recon;)

alert tcp any any -> $HOME_NET any (msg:"Stephanie Salgado... TCP scan detected.";sid:10000005;rev:2;)

```

```

stephanie@ubuntu:~$ sudo nmap -sS 192.168.200.128
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-11-14 14:37 CST
Nmap scan report for ubuntu (192.168.200.128)
Host is up (0.0000020s latency).
All 1000 scanned ports on ubuntu (192.168.200.128) are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 1 IP address (1 host up) scanned in 0.08 seconds

```

```

11/14-14:39:10.104438  [**] [1:1000001:0] Stephanie Salgado... IP packet is detected. [**] [Priority: 0] {UDP} 192.168.200.1:52392 -> 239.255.255.250:1900
11/14-14:39:34.141617  [**] [1:10000005:2] Stephanie Salgado... TCP scan detected. [**] [Priority: 0] {TCP} 34.193.174.221:443 -> 192.168.200.128:40566
11/14-14:39:34.141617  [**] [1:1000001:0] Stephanie Salgado... IP packet is detected. [**] [Priority: 0] {TCP} 34.193.174.221:443 -> 192.168.200.128:40566
11/14-14:39:34.141621  [**] [1:10000005:2] Stephanie Salgado... TCP scan detected. [**] [Priority: 0] {TCP} 34.193.174.221:443 -> 192.168.200.128:40566
11/14-14:39:34.141621  [**] [1:1000001:0] Stephanie Salgado... IP packet is detected. [**] [Priority: 0] {TCP} 34.193.174.221:443 -> 192.168.200.128:40566
11/14-14:39:34.141621  [**] [1:10000005:2] Stephanie Salgado... TCP scan detected. [**] [Priority: 0] {TCP} 34.193.174.221:443 -> 192.168.200.128:40566
11/14-14:39:34.141621  [**] [1:1000001:0] Stephanie Salgado... IP packet is detected. [**] [Priority: 0] {TCP} 34.193.174.221:443 -> 192.168.200.128:40566
11/14-14:39:34.141622  [**] [1:10000005:2] Stephanie Salgado... TCP scan detected. [**] [Priority: 0] {TCP} 162.159.61.4:443 -> 192.168.200.128:59120
11/14-14:39:34.141622  [**] [1:1000001:0] Stephanie Salgado... IP packet is detected. [**] [Priority: 0] {TCP} 162.159.61.4:443 -> 192.168.200.128:59120
11/14-14:39:34.141622  [**] [1:10000005:2] Stephanie Salgado... TCP scan detected. [**] [Priority: 0] {TCP} 162.159.61.4:443 -> 192.168.200.128:59120

```

Here I'm adding a 4th rule to the "test.rule" file. I then used "sudo nmap -sS 192.168.200.128" to perform a TCP scan that checks for open ports on the target IP address (the \$HOME_NET) without completing the full connection. As shown, this rule detects the TCP scan.

Step 3: Detecting each visit to www.google.com

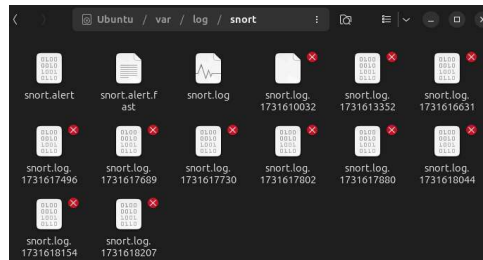
```
stephanie@ubuntu:~$ sudo cat /etc/snort/rules/local.rules
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
# -----
# LOCAL RULES
# -----
# This file intentionally does not come with signatures.  Put your local
# additions here.
alert ip any any -> any any (msg: "Stephanie Salgado... IP packet is detected.";
sid:1000001;)

# I added this rule to detect each visit to www.google.com
alert tcp $HOME_NET any -> any 80 (msg:"Stephanie Salgado... Access to Google de
tected.";reference:url,www.google.com;sid:1000000;rev:1;)
```

```
stephanie@ubuntu:~$ sudo snort -q -A console -c /etc/snort/snort.conf
11/14-15:40:27.607881  [**] [1:1000000:1] Stephanie Salgado... Access to Google
detected. [**] [Priority: 0] {TCP} 192.168.200.128:33012 -> 142.250.114.99:80
11/14-15:41:10.285780  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [
Classification: Detection of a Network Scan] [Priority: 3] {UDP} 192.168.200.1:5
2392 -> 239.255.255.250:1900
11/14-15:41:12.673959  [**] [1:1000000:1] Stephanie Salgado... Access to Google
```

```
stephanie@ubuntu:~$ curl -v www.google.com
* Host www.google.com:80 was resolved.
* IPv6: 2607:f8b0:4023:1004::6a, 2607:f8b0:4023:1004::63, 2607:f8b0:4023:1004::6
7, 2607:f8b0:4023:1004::93
* IPv4: 142.251.116.99, 142.251.116.104, 142.251.116.106, 142.251.116.103, 142.2
51.116.105, 142.251.116.147
* Trying 142.251.116.99:80...
* Connected to www.google.com (142.251.116.99) port 80
> GET / HTTP/1.1
> Host: www.google.com
```

Finally, I added another rule to the “local.rules” file in order to detect each visit to “www.google.com”. I used the same command to run Snort in IDS mode. At first, I had trouble finding any of the alerts for the rule I had added. I edited the “local.rules” file again and commented out the other rule I had in there. Then, I restarted Snort and ran it using the IDS mode command another time. This time, I was able to get alerts when visiting “www.google.com” using the “curl -v www.google.com” command.



Ultimately, I wasn't able to verify that the alert was triggered through my Snort logs because I wasn't able to view them, but the console confirmed the alert was triggered when it displayed the message for the google rule.

Questions:

1. In step 1, how did you modify the config file to make it work?

My config file was originally working, but when I added the path to the provided "test.rules" file, it seemed to have an error. That was easily fixed by going into the file and looking at the line giving the error, in this case, the problem was that "recon;)" from the third rule was on a new line and separated from "classtype:attempted-".

2. In step 2, describe the two attack signatures you chose and explain the corresponding rules against them.

In step 2, I simply wanted to create a rule to detect a TCP scan. I was able to do this using the 4th rule in the "test.rules" file and using nmap for the scan.

3. How did you attempt to trigger the alert? How did snort process your requests?

I tried using Firefox but I wasn't successful so then I used "curl". After that, Snort was able to view each time I accessed "www.google.com".

Both questions 2 & 3 weren't supposed to be answered (mentioned in the lab meeting), but I wanted to try answering parts of them if possible.

Summary

For this lab, I used an Ubuntu 24 VM to install, set up, and run Snort, learning how to use it in different modes. After installing Snort, I checked the configuration file and used a command to validate it. Then, I used Snort in sniffing mode, where it captured packets sent by a ping command. After that, I used it in IDS mode, where it triggered alerts based on rules in “local.rules” and “test.rules”. I was able to make a rule to detect IP packets and confirmed Snort was working correctly by restarting the service. I also added a rule to detect a TCP scan using Nmap, which triggered alerts. Finally, I created a rule to detect visits to “www.google.com” and confirmed it worked by using curl to trigger the alert in the console. Through this lab, I learned to configure, test, and customize Snort to detect specific network traffic, giving me a decent understanding of it.